CASE STUDY

Building a Store Inventory System

How a student used arrays, structures, and three algorithms to solve a real shop-management problem in C

25CAT-151 :: DATA STRUCTURES USING C

1. The Problem: A Shop in a Mess

Picture a small grocery store. The owner keeps a handwritten notebook with every product — what it is, how much it costs, and how many packets are left on the shelf. Every morning she scans through pages of crossed-out entries and scribbled corrections just to figure out what needs restocking. When a customer asks the price of something, she has to flip through the whole notebook.

Sound familiar? Most small stores start exactly like this. The notebook works — until there are 80 products, three staff members adding entries in different handwriting, and nobody can find the item they are looking for.

This case study is about a student who was asked to solve this exact problem — not for a real shop (yet), but as a lab exercise for the Data Structures course (25CAT-151). The task: build a simple inventory system in C that lets someone add products, look them up by ID, delete old ones, and sort the list by price or by how much stock is left. No fancy database. No internet. Just C and the concepts learned in class.

2. What the System Needed to Do

Before writing a single line of code, the student sat down and listed every thing the system had to handle. This step — called requirement gathering — is something every software project does, big or small.

Here is what the final list looked like:

- Add a new product — store its ID, name, price, and how many units are in stock.
- Show all products — print every record in a clean, readable table.
- Find a product — let the user type an ID and get back that product's details instantly.
- Remove a product — delete a record when a product is discontinued.
- Sort by price — arrange the whole list from cheapest to most expensive.
- Sort by quantity — arrange the list from lowest stock to highest, so you can spot what needs restocking first.
- Exit cleanly — let the user quit without losing the session's work unexpectedly.

The system would run in the terminal (the black command-line window). No buttons, no mouse — just text typed by the user and text printed back by the program. This is called a console application, and it is a great environment for learning because there are no visual distractions: the logic is front and centre.

3. How the Data Was Stored

3.1 Choosing the Right Structure

The first real design decision was: how do we keep all the product records in memory while the program is running?

In C, when you want to store several pieces of information about one thing together, you use a struct. Think of a struct like a row in a table — one row for each product, with columns for ID, name, price, and quantity. The student defined it like this:

```
typedef struct {
    int   id;          /* e.g. 101, 102, 103 ...              */
    char  name[50];    /* e.g. "Apple Juice", "Bread" ...     */
    float price;       /* e.g. 45.50, 25.00 ...               */
    int   quantity;    /* e.g. 30 units, 50 units ...         */
} Product;
```

Once the structure was defined, an array of these structs served as the inventory — like a spreadsheet with 100 rows ready to be filled:

```
#define MAX_PRODUCTS 100
Product inventory[MAX_PRODUCTS];
Int count = 0;
```

The variable count is the real workhorse here. It starts at zero and goes up by one every time a product is added. It goes down by one every time a product is deleted. Every function in the program uses count to know where the valid data ends — so nothing ever accidentally reads an empty slot.

3.2 Why an Array and Not Something Else?

Students often ask: why not use a linked list? It is a fair question. A linked list can grow without a size limit, which sounds better. But for this project, an array was the right choice for three reasons:

1. Simplicity — array indexing (inventory[i]) is straightforward. A linked list needs pointers, malloc, and careful memory management, which adds complexity without adding value at this scale.
2. Sorting — both Selection Sort and Insertion Sort are designed to work on arrays. Adapting them to a linked list would require extra code.
3. The size is known — a real shop rarely has more than a few hundred product types. 100 is more than enough for a lab exercise, and the constant MAX_PRODUCTS makes it trivial to increase later.

4. The Algorithms Behind the Scenes

Three core algorithms power the system. Each one was chosen because it matched the problem it was solving.

4.1 Finding a Product — Linear Search

When the user types an ID and hits Enter, the program needs to find that product quickly. The approach used here is called Linear Search — which is exactly as straightforward as it sounds. The program starts at the first product, checks if the ID matches, then moves to the second, then the third, and so on, until it either finds a match or runs out of records.

Here is the core of that logic in plain words:

"Go through every product one by one. As soon as you find the one with the right ID, show it and stop. If you reach the end without finding it, say not found."

In code:

```
for (int i = 0; i < count; i++) {
    if (inventory[i].id == id) {
        /* Found it! Print and return immediately */
        printf("Name: %s | Price: %.2f | Qty: %d\n",
               inventory[i].name, inventory[i].price,
               inventory[i].quantity);
        return;
    }
}
printf("Product not found.\n");
```

In the worst case — when the product is not in the list at all — every single record gets checked. For 100 products that is 100 comparisons. That is perfectly fine. Linear Search is simple, correct, and fast enough for any inventory a small shop would have.

4.2 Sorting by Price — Selection Sort

When the user wants to sort by price, the program uses Selection Sort. Here is the idea in everyday language:

"Look at all the products. Find the cheapest one. Move it to the front. Now ignore the front and look at the rest. Find the cheapest of those. Move it to the second position. Keep going until everything is in order."

Each pass through the list finds the minimum-price product in the unsorted part and swaps it into place. After n passes, the list is fully sorted from cheapest to most expensive.

What makes Selection Sort particularly well-suited here is that it performs very few swaps — at most one per pass. And swapping two products means copying an entire struct (ID + name + price + quantity). Fewer swaps means less copying, which means faster execution when records are large.

```
for (int i = 0; i < count - 1; i++) {
    int minIdx = i;
    for (int j = i + 1; j < count; j++)
        if (inventory[j].price < inventory[minIdx].price)
            minIdx = j;
    if (minIdx != i) {
```

```
        Product tmp        = inventory[i];
        inventory[i]       = inventory[minIdx];
        inventory[minIdx]  = tmp;
    }
}
```

4.3 Sorting by Quantity — Insertion Sort

Sorting by quantity uses Insertion Sort — a different strategy that is easier to see if you think about playing cards.

"Imagine you are holding a hand of cards sorted from low to high. Someone deals you a new card. You slide it into the right position without shuffling the whole hand. Do that for every card, one at a time, until all cards are sorted."

Insertion Sort works the same way. It takes each product one at a time and slides it backwards through the already-sorted portion until it sits in the right spot.

```
for (int i = 1; i < count; i++) {
    Product key = inventory[i];
    int j = i - 1;
    while (j >= 0 && inventory[j].quantity > key.quantity) {
        inventory[j + 1] = inventory[j];
        j--;
    }
    inventory[j + 1] = key;
}
```

Insertion Sort has a nice bonus: if the inventory is already nearly sorted — say, you just added a couple of new products — it finishes very quickly, almost instantly. This makes it a smart choice for the quantity sort, since stock levels change often and small adjustments are common.

4.4 Algorithm Comparison

Here is a quick side-by-side look at how the three algorithms compare:

Algorithm	Used For	Best Case	Worst Case	Why It Was Chosen
Linear Search	Finding a product by ID	O(1) — found first	O(n) — not found	Simple, works on unsorted data
Selection Sort	Sort by Price	$O(n^2)$	$O(n^2)$	Minimises costly struct swaps
Insertion Sort	Sort by Quantity	O(n) — nearly sorted	$O(n^2)$	Fast when data is mostly ordered

5. Building It Step by Step

The student did not write the whole program at once. It was built one function at a time, testing each piece before moving on. This is a good habit — writing everything and then testing is a recipe for confusion.

Step 1 — Define the Data

First came the struct and the global array. Nothing could be written until it was clear what a "product" looked like in the program's memory. Getting this right early saved a lot of refactoring later.

Step 2 — Write the Menu Loop

The second step was to write the main() function with a menu that showed options and read the user's choice — even before any of the actual operations worked. This gave a running, interactive shell to test against from the beginning.

```
do {
    showMenu();
    scanf("%d", &choice);
    switch (choice) {
        case 1: addProduct();      break;
        case 2: displayProducts(); break;
        case 3: searchProduct();   break;
        case 4: deleteProduct();   break;
        case 5: sortByPrice();     break;
        case 6: sortByQuantity();  break;
        case 7: printf("Goodbye!\n"); break;
        default: printf("Invalid option.\n");
    }
} while (choice != 7);
```

Step 3 — Add Products

Once the menu was working, the add function was written. This was the trickiest part of input handling. Mixing scanf (for numbers) and fgets (for the name) in the same program causes problems if you are not careful — scanf leaves a newline in the input buffer that fgets then reads as an empty line. The fix is a single getchar() call between the two.

```
scanf("%d", &p.id);
getchar();
fgets(p.name, sizeof(p.name), stdin);
p.name[strcspn(p.name, "\n")] = '\0';
```

Once this was working correctly, the student could add products and immediately check them with the display function.

Step 4 — Search and Delete

Search and delete were written next. Both start with the same linear scan. Delete has one extra step: after finding the record, it shifts everything after it one slot to the left. This is the cleanest way to remove an element from an array without leaving a hole in the middle.

Step 5 — Sort

The two sort functions came last, because they are the most complex and they depend on display() to show the result. Writing them last meant that when a sort produced the wrong output, the display code was already known to be correct — which made debugging much faster.

6. The System in Action

Once the program was complete, the student ran it with a set of sample products to make sure everything worked end-to-end. Here is a walkthrough of that session.

Starting the Program

The user is greeted with a numbered menu:

```
==============================================================
          SIMPLE INVENTORY MANAGEMENT SYSTEM
==============================================================
  1. Add Product
  2. Display All Products
  3. Search Product (by ID)
  4. Delete Product (by ID)
  5. Sort by Price    [Selection Sort]
  6. Sort by Quantity [Insertion Sort]
  7. Exit
--------------------------------------------------------------
  Enter your choice:
```

Adding Three Products

The user adds three products. Each addition is confirmed with the updated count:

```
Enter Product ID   : 101
Enter Product Name : Apple Juice
Enter Price        : 45.50
Enter Quantity     : 30
[OK] Product added. Total products: 1

Enter Product ID   : 102
Enter Product Name : Bread
Enter Price        : 25.00
Enter Quantity     : 50
[OK] Product added. Total products: 2

Enter Product ID   : 103
Enter Product Name : Milk
Enter Price        : 35.75
Enter Quantity     : 20
[OK] Product added. Total products: 3
```

Viewing the Inventory

The full list is printed in a padded table so columns always line up, no matter how long the product names are:

```
--- Inventory List (3 product(s)) ---
--------------------------------------------------------------
| ID     | Name                  | Price      | Qty      |
--------------------------------------------------------------
| 101    | Apple Juice           | 45.50      | 30       |
```

```
| 102    | Bread                | 25.00      | 50       |
| 103    | Milk                 | 35.75      | 20       |
------------------------------------------------------------
```

Searching for a Product

The user looks up Milk by its ID. The program scans from ID 101, skips it, checks 102, skips it, finds 103, and prints:

```
Enter Product ID to search: 103

[OK] Product found:
  ID       : 103
  Name     : Milk
  Price    : 35.75
  Quantity : 20
```

Sorting by Price

After sorting by price, the cheapest product (Bread at 25.00) moves to the top:

```
[OK] Inventory sorted by price (ascending) — Selection Sort.
------------------------------------------------------------
| 102    | Bread                | 25.00      | 50       |
| 103    | Milk                 | 35.75      | 20       |
| 101    | Apple Juice          | 45.50      | 30       |
------------------------------------------------------------
```

Sorting by Quantity (Stock Level)

Sorting by quantity puts Milk first — a useful view for restocking, since Milk has the lowest stock:

```
[OK] Inventory sorted by quantity (ascending) — Insertion Sort.
------------------------------------------------------------
| 103    | Milk                 | 35.75      | 20       |
| 101    | Apple Juice          | 45.50      | 30       |
| 102    | Bread                | 25.00      | 50       |
------------------------------------------------------------
```

7. Challenges and How They Were Solved

No project goes perfectly from start to finish. Here are the three biggest problems encountered during development and how they were fixed.

Challenge 1 — The Name Field Ate Empty Input

The first version of addProduct() used scanf for everything, including the name. But scanf stops reading at a space, so "Apple Juice" would be stored as just "Apple" — and the word "Juice" would be left in the input buffer, causing the next scanf to read it as the next product's ID.

Fix: Switch the name field to fgets, which reads the entire line including spaces. Add getchar() before it to consume the newline that the previous scanf left behind. Add strcspn to remove the newline that fgets adds at the end of the string. Three small changes, problem solved.

Challenge 2 — The Sort Changed the Wrong Elements

The first attempt at Selection Sort had a bug: the swap happened every pass, even when the minimum was already in the right place. This was harmless for correctness (the product swapped with itself) but slow.

Fix: Add a simple check — if (minIdx != i) — before the swap. If the minimum is already at position i, skip the swap entirely. This is a standard optimisation and makes the code easier to read too.

Challenge 3 — Deleting a Product Left a Gap

The first delete attempt just set the deleted slot to zeros. But then the display loop would show a blank row in the middle of the table, and the count was wrong.

Fix: Instead of zeroing the slot, shift every record after it one position to the left, then decrement count. The array stays dense — no gaps, no blank rows, no confusion.

8. What This Project Teaches

On the surface, this is a C programming exercise. But the lessons it contains go well beyond syntax.

Data and structure come first

Before writing a single function, the student had to decide how data would be stored. Getting the struct right — and choosing an array over more complex structures — made every other part of the project simpler. This is true in any software project, at any scale.

The right algorithm for the right job

All three algorithms used here — Linear Search, Selection Sort, Insertion Sort — are called naive or elementary. They are not the fastest algorithms ever invented. But they are correct, easy to understand, easy to implement, and fast enough for a small dataset. Choosing a simple algorithm that works is almost always better than reaching for a complex one before you need it.

Modular code is easier to fix

Because each operation was in its own function, debugging was much easier. When the sort was wrong, only the sort function needed to be looked at. When the input was broken, only the add function needed to be changed. Nothing else was affected. This is the principle of separation of concerns — keeping unrelated things apart — and it is one of the most important ideas in software engineering.

Testing step by step saves time

Building one feature at a time and testing it before moving on meant that when something broke, the break was always in the most recently written code. This is far easier to debug than writing everything and then running the whole program for the first time and finding ten things wrong at once.

9. What Could Be Added Next

This project is an MVP — Minimum Viable Product. It does exactly what it needs to do and nothing more. But if someone wanted to take it further, here are natural next steps:

- Save data to a file — right now, everything is lost when the program closes. Adding file read/write using fopen and fprintf would fix that. A follow-up version of this project already does this using a CSV file.
- Check for duplicate IDs — the current system does not stop you from adding two products with the same ID. A quick scan before insertion would catch this.
- Add an Edit option — if a price changes, right now the only way to update it is to delete the product and re-add it. An edit function would be more user-friendly.
- Use a linked list — replacing the array with a dynamically allocated linked list would remove the 100-product limit and make insertions and deletions faster.
- Calculate total inventory value — a simple loop multiplying price × quantity for every product and summing the results would give the total value of all stock, which is useful for accounting.

10. Conclusion

The Simple Inventory Management System started as a list of requirements written on a piece of paper and ended as a working, tested C program that can manage a small store's products.

Along the way, it demonstrated something important: the data structure concepts taught in class — arrays, structs, search, and sorting — are not abstract ideas invented for examinations. They are tools that solve real problems. The array-of-structures kept product records organised. Linear Search found them quickly. Selection Sort kept the price view tidy. Insertion Sort kept the stock view efficient.

Every function in the program does one thing and does it well. Every design decision — choosing an array over a linked list, choosing fgets over scanf, shifting instead of zeroing on delete — was made for a reason that can be explained clearly. That is what separates code that just runs from code that is actually understood.

This project is a foundation. The shop owner in the opening story may still be using her notebook — but a system like this is the first real step towards something better.